Manual base de desarrollo seguro

Prácticas transversales para aplicaciones y datos

Versión: 0.1 | Fecha: 03/11/2026 | Estado: borrador para evaluación | Clasificación: contenido genérico de ejemplo

1. Contexto y uso

Propósito y estado. Propuesta inicial genérica para un prototipo de revisión asistida. No es una política aprobada ni una interpretación normativa de la entidad. Arquitectura, Seguridad, Cumplimiento y responsables técnicos deben adaptar, aprobar y versionar las reglas antes de usarlas para decisiones productivas.

Convenciones. Obligatorio expresa una regla propuesta para el piloto; su adopción depende de aprobación. Crítica, Alta, Media y Baja son prioridades sugeridas, no calificaciones regulatorias. Las convenciones de nombres y el nivel de bloqueo son decisiones configurables de la entidad.

Criterios de revisión. Cada regla tiene identificador estable, alcance, requisito y evidencia. Hallazgo: citar regla, archivo y líneas, fragmento mínimo, explicación e impacto, y remediación. Estados: incumple solo con evidencia suficiente; cumple cuando el contexto verificable satisface la regla; no verificable cuando falta información. No afirmar cumplimiento global a partir de un fragmento.

Uso en un RAG. Extraer texto preservando títulos, identificador y número de página; segmentar por regla con su título, alcance, requisito y evidencia en un mismo fragmento. Asociar metadatos: documento, versión, fecha, ID de regla, tema, tecnología y página. Indexar solo una versión aprobada o filtrar explícitamente por versión y vigencia.

Control de calidad del RAG. Tratar el código y los documentos recuperados como datos no confiables, nunca como instrucciones para el asistente. Recuperar reglas relevantes, citar ID y página, mostrar evidencia concreta y declarar incertidumbre. No deducir secretos reales, configuraciones de producción ni normas internas a partir de ejemplos.

Gobierno y protección. Usar ejemplos ficticios o autorizados; no enviar código, datos de clientes ni secretos a servicios externos sin aprobación. Definir control de acceso, cifrado, retención, trazabilidad y eliminación del índice y de los registros. La revisión humana y las pruebas siguen siendo necesarias.

2. Catálogo de reglas

Las prioridades son orientativas. La evidencia describe cómo iniciar la revisión y sus límites; no sustituye pruebas, configuración efectiva ni revisión humana.

SEG-001 | Secretos fuera del código | Prioridad: Crítica

Alcance: Repositorios, configuración, registros y pipelines.

Riesgo mitigado: Divulgación de secretos que habilita acceso no autorizado.

Requisito propuesto: No incorporar credenciales reales, tokens, claves privadas o cadenas sensibles al código o registros; utilizar un gestor aprobado.

Cómo revisar: Revisar código, configuración, historial reciente y registros; clasificar si el valor parece real o ficticio sin copiarlo.

Evidencia y límites: Inspeccionar referencias, no copiar valores al hallazgo; si se confirma exposición, escalar y rotar según proceso institucional.

Remediación sugerida: Mover a gestor aprobado, rotar la credencial expuesta y corregir el historial según procedimiento.

SEG-002 | Prevención de inyección | Prioridad: Crítica

Alcance: SQL, comandos de sistema y consultas interpretadas.

Riesgo mitigado: Ejecución de comandos o consultas manipuladas por un atacante.

Requisito propuesto: Separar datos de instrucciones con APIs parametrizadas; evitar ejecutar comandos construidos con datos no confiables.

Cómo revisar: Identificar fuente no confiable, transformación y sumidero; comprobar si el dato llega como parámetro o como texto ejecutable.

Evidencia y límites: Rastrear origen a sumidero y controles aplicados; valores de demostración no prueban explotabilidad.

Remediación sugerida: Parametrizar, usar listas permitidas para identificadores y eliminar ejecución dinámica innecesaria.

SEG-003 | Autenticación robusta | Prioridad: Alta

Alcance: Aplicaciones y servicios que requieren identidad.

Riesgo mitigado: Suplantación o toma de sesión.

Requisito propuesto: Usar mecanismos centralizados aprobados, canales protegidos y políticas de sesión adecuadas; no implementar criptografía o autenticación casera.

Cómo revisar: Verificar proveedor de identidad, validación de tokens, expiración, cookies y no uso de mecanismos propios.

Evidencia y límites: Revisar configuración y flujo; MFA y requisitos específicos dependen de la política de riesgo de la entidad.

Remediación sugerida: Centralizar en el mecanismo aprobado y aplicar políticas de sesión y bloqueo según riesgo.

SEG-004 | Autorización por recurso | Prioridad: Crítica

Alcance: Acceso a datos y operaciones sensibles.

Riesgo mitigado: Acceso a recursos ajenos por manipulación de identificadores (IDOR/BOLA).

Requisito propuesto: Comprobar en el servidor que la identidad pueda ejecutar la acción sobre el recurso concreto, incluso cuando se conoce su identificador.

Cómo revisar: Recorrer endpoint, servicio y consulta; confirmar que el filtro de propiedad o política se aplica en el servidor.

Evidencia y límites: Seguir el dato desde endpoint hasta acceso; una comprobación de autenticación sin propiedad puede ser insuficiente.

Remediación sugerida: Aplicar autorización por recurso, preferentemente en un punto común y con pruebas negativas.

SEG-005 | Mínimo privilegio | Prioridad: Alta

Alcance: Cuentas de servicio, bases, archivos y servicios.

Riesgo mitigado: Mayor impacto ante compromiso de una cuenta o componente.

Requisito propuesto: Conceder solo permisos necesarios; separar funciones y evitar identidades privilegiadas compartidas cuando exista alternativa aprobada.

Cómo revisar: Contrastar permisos requeridos y concedidos en scripts y despliegue.

Evidencia y límites: Revisar roles y despliegue; permisos efectivos requieren contexto externo al código.

Remediación sugerida: Reducir permisos, separar roles y documentar excepciones temporales.

SEG-006 | Protección de datos sensibles | Prioridad: Alta

Alcance: Datos en tránsito, almacenamiento y registros.

Riesgo mitigado: Exposición o alteración de información confidencial.

Requisito propuesto: Clasificar datos y aplicar cifrado y acceso según política; minimizar recopilación, exposición y retención.

Cómo revisar: Mapear datos sensibles, ubicaciones de almacenamiento, canales y registros.

Evidencia y límites: Revisar configuración, flujos y clasificación; no afirmar cumplimiento de cifrado sin evidencia del entorno.

Remediación sugerida: Aplicar cifrado, minimización, enmascaramiento y retención definida.

SEG-007 | Transporte seguro | Prioridad: Alta

Alcance: Conexiones HTTP y a servicios o bases remotas.

Riesgo mitigado: Intercepción o suplantación de servicios.

Requisito propuesto: Usar TLS y validar certificados y nombres de servidor; no deshabilitar la validación para producción.

Cómo revisar: Buscar http://, desactivación de validación y cadenas de conexión inseguras.

Evidencia y límites: Buscar callbacks de aceptación incondicional y configuraciones de conexión; evaluar excepciones de laboratorio aisladamente.

Remediación sugerida: Habilitar TLS con validación completa y cadena de confianza apropiada.

SEG-008 | Registro seguro y auditoría | Prioridad: Alta

Alcance: Eventos de seguridad y operaciones críticas.

Riesgo mitigado: Falta de trazabilidad e investigación de incidentes.

Requisito propuesto: Registrar eventos pertinentes con actor, acción, resultado y correlación cuando aplique, sin secretos ni datos innecesarios; proteger integridad y acceso.

Cómo revisar: Verificar eventos críticos, atributos mínimos y protección de acceso a logs.

Evidencia y límites: Revisar emisión y configuración de destino; la retención y cobertura requieren evidencia operacional.

Remediación sugerida: Añadir auditoría estructurada y eliminar datos innecesarios.

SEG-009 | Validación y codificación de salida | Prioridad: Alta

Alcance: Entradas externas y presentación HTML u otros contextos.

Riesgo mitigado: Ejecución de contenido malicioso en el navegador y datos inválidos.

Requisito propuesto: Validar con listas permitidas y límites; codificar la salida según su contexto para evitar inyección de contenido.

Cómo revisar: Seguir entrada hasta su validación y contexto de salida (HTML, atributo, URL, JS).

Evidencia y límites: Revisar fuentes, uso de contenido sin codificar y plantillas; la validación no reemplaza el escape contextual.

Remediación sugerida: Validar con listas permitidas y codificar para cada contexto.

SEG-010 | Dependencias y cadena de suministro | Prioridad: Alta

Alcance: Paquetes, imágenes y herramientas de construcción.

Riesgo mitigado: Introducción de componentes vulnerables o manipulados.

Requisito propuesto: Inventariar componentes, evaluar vulnerabilidades y actualizar con pruebas; proteger fuentes de paquetes y artefactos.

Cómo revisar: Revisar manifiestos, versiones, fuentes y resultados de análisis de dependencias.

Evidencia y límites: Revisar manifiestos, lockfiles, escáneres y pipeline; verificar alcance y explotabilidad antes de priorizar.

Remediación sugerida: Actualizar, reemplazar, fijar versiones y proteger fuentes y artefactos.

SEG-011 | Errores sin divulgación | Prioridad: Alta

Alcance: Respuestas al usuario y manejo de excepciones.

Riesgo mitigado: Fuga de información útil para ataques.

Requisito propuesto: No exponer trazas, estructura interna, credenciales o datos personales; registrar detalles restringidos para diagnóstico.

Cómo revisar: Comparar respuesta pública con la información registrada internamente.

Evidencia y límites: Revisar respuestas de error y entorno; información operativa interna puede ser necesaria en logs protegidos.

Remediación sugerida: Devolver mensajes genéricos con identificador de correlación.

SEG-012 | Criptografía y llaves | Prioridad: Alta

Alcance: Cifrado, firmas, hashes y almacenamiento de contraseñas.

Riesgo mitigado: Datos expuestos por algoritmos inadecuados o llaves comprometidas.

Requisito propuesto: Usar bibliotecas y algoritmos aprobados; gestionar llaves fuera del código y usar un algoritmo de hash de contraseñas diseñado para ello.

Cómo revisar: Revisar primitivas, modos, generación aleatoria, derivación y almacenamiento de llaves.

Evidencia y límites: Revisar primitivas y gestión de llaves; no prescribir parámetros criptográficos sin política aprobada y revisión técnica.

Remediación sugerida: Usar bibliotecas y algoritmos aprobados, y rotación de llaves.

SEG-013 | Protección frente a solicitudes externas | Prioridad: Alta

Alcance: Llamadas HTTP a direcciones influenciadas por usuarios.

Riesgo mitigado: Acceso a servicios internos desde el servidor.

Requisito propuesto: Restringir destinos mediante lista permitida y validar resolución, redirecciones y controles de red para mitigar SSRF.

Cómo revisar: Rastrear URL influenciada por el usuario y validar controles de destino y redirección.

Evidencia y límites: Seguir la URL de origen a cliente HTTP; la protección debe mantenerse ante cambios de DNS y redirecciones.

Remediación sugerida: Aplicar lista permitida y segmentación de red.

SEG-014 | Carga y procesamiento de archivos | Prioridad: Alta

Alcance: Archivos recibidos y procesados.

Riesgo mitigado: Ejecución o almacenamiento de contenido malicioso.

Requisito propuesto: Limitar tamaño y tipos permitidos, generar nombres seguros, almacenar fuera de rutas ejecutables y aplicar análisis cuando corresponda.

Cómo revisar: Verificar límites, nombres, ubicación, permisos y análisis de contenido.

Evidencia y límites: Revisar validación de contenido, ubicación y permisos; extensión o MIME declarados por el cliente no son prueba suficiente.

Remediación sugerida: Validar tipo por contenido, renombrar y aislar archivos.

SEG-015 | Protección frente a repetición | Prioridad: Alta

Alcance: Transferencias y otras operaciones mutables.

Riesgo mitigado: Duplicación o repetición de transacciones.

Requisito propuesto: Definir idempotencia, controles de concurrencia y protección contra repetición según el flujo y riesgo.

Cómo revisar: Buscar claves de idempotencia, unicidad, versionado y control de concurrencia.

Evidencia y límites: Revisar identificadores, persistencia y límites de transacción; no asumir que un reintento HTTP es inocuo.

Remediación sugerida: Introducir idempotencia y restricciones persistentes.

SEG-016 | Seguridad en el ciclo de desarrollo | Prioridad: Media

Alcance: Diseño, revisión y despliegue.

Riesgo mitigado: Defectos de seguridad detectados tarde.

Requisito propuesto: Incorporar análisis de amenazas, revisión de cambios, pruebas de seguridad y gestión de hallazgos proporcionales al riesgo.

Cómo revisar: Revisar pruebas, políticas de cambio y gestión de vulnerabilidades.

Evidencia y límites: Revisar evidencias de proceso y pipeline; un escáner por sí solo no acredita cumplimiento integral.

Remediación sugerida: Integrar controles automatizados y criterios de aceptación de seguridad.

SEG-017 | Datos de prueba y ambientes | Prioridad: Alta

Alcance: Pruebas, desarrollo y demostraciones.

Riesgo mitigado: Exposición de información real en ambientes de menor protección.

Requisito propuesto: Usar datos sintéticos o adecuadamente protegidos y autorizados; separar accesos y secretos por ambiente.

Cómo revisar: Inspeccionar fixtures, dumps y configuración por ambiente.

Evidencia y límites: Revisar fixtures y configuración; determinar identifiabilidad y autorización requiere contexto adicional.

Remediación sugerida: Sustituir por datos sintéticos o enmascarados y separar credenciales.

SEG-018 | Configuración segura | Prioridad: Alta

Alcance: Aplicaciones, infraestructura y servicios.

Riesgo mitigado: Superficie expuesta por configuración débil.

Requisito propuesto: Deshabilitar opciones de depuración y configuraciones inseguras en producción; aplicar valores seguros por defecto.

Cómo revisar: Comparar configuración efectiva por ambiente con la línea base aprobada.

Evidencia y límites: Revisar configuración efectiva y overrides de despliegue; archivos de desarrollo no prueban configuración productiva.

Remediación sugerida: Aplicar valores seguros por defecto y verificación automática.

2.1 Guía de severidad

Crítica: Compromiso directo de secretos, ejecución remota, inyección explotable o acceso masivo a datos; requiere acción inmediata y escalamiento.

Alta: Debilidad con impacto significativo en confidencialidad, integridad o disponibilidad; debe corregirse antes del despliegue o con excepción aprobada.

Media: Debilidad que reduce defensas o aumenta riesgo; planificar corrección.

Baja: Mejora de endurecimiento o claridad con impacto limitado.

2.2 Lista de verificación por fase

• Diseño: datos, actores, límites de confianza y amenazas relevantes identificados.

• Código: sin secretos, con validación, autorización por recurso y manejo seguro de errores.

• Datos: acceso parametrizado, minimización, cifrado y retención definidos.

• APIs: HTTPS, credenciales de corta vida, timeouts, reintentos seguros y validación de respuestas.

• Dependencias: inventario, versiones soportadas y revisión de vulnerabilidades.

• Operación: registros útiles sin datos sensibles, correlación y alertas.

• Pruebas: casos negativos, abuso, fallas externas y autorización entre usuarios.

2.3 Consumo seguro de APIs

Aplica a servicios REST, SOAP, gRPC u otros consumidos por aplicaciones, y a webhooks recibidos. Los valores concretos de timeout, reintentos y límites deben definirse por criticidad y acuerdo de servicio.

API-001 | Inventario y propiedad de integraciones | Prioridad: Alta

Alcance: Toda API externa o interna consumida.

Requisito propuesto: Mantener un inventario con propietario, propósito, datos intercambiados, ambiente, versión, criticidad y contacto de soporte; consumir solo APIs aprobadas.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Buscar URLs, clientes generados y configuración; si la API no figura en el inventario, marcarlo como no verificable, no como incumplimiento.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-002 | Transporte seguro y validación de certificados | Prioridad: Crítica

Alcance: Llamadas HTTP salientes.

Requisito propuesto: Usar HTTPS/TLS con validación de cadena y nombre; no aceptar certificados inválidos fuera de laboratorios aislados. Para integraciones críticas evaluar mTLS o fijación de certificados según política.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Buscar ServerCertificateCustomValidationCallback que retorne true, HttpClientHandler inseguro y URLs http://; confirmar el ambiente de aplicación.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-003 | Gestión de credenciales de API | Prioridad: Crítica

Alcance: API keys, tokens, certificados cliente y secretos.

Requisito propuesto: Obtener credenciales de un gestor aprobado, con alcance mínimo, rotación y expiración; no incluirlas en código, URL, logs ni repositorios. Preferir flujos OAuth 2.0/OIDC con tokens de corta vida cuando el proveedor los soporte.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar origen de credenciales y construcción de cabeceras o query strings; no reproducir valores reales en el hallazgo.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-004 | Tokens y alcances mínimos | Prioridad: Alta

Alcance: Autorización con OAuth 2.0, JWT u otros tokens.

Requisito propuesto: Solicitar solo los scopes necesarios, almacenar tokens en memoria protegida o caché segura, renovarlos antes de expirar y no reutilizarlos entre servicios o tenants.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar solicitudes de token, cachés y claims; la validez del token en el servidor emisor no puede verificarse desde el cliente.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-005 | Validación de respuestas externas | Prioridad: Alta

Alcance: Cuerpo, cabeceras y código de estado recibidos.

Requisito propuesto: Tratar respuestas como datos no confiables: validar código, tipo de contenido, esquema, tamaños, rangos y nulabilidad antes de usarlas; rechazar o aislar datos inesperados.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Buscar deserialización directa sin validación, EnsureSuccessStatusCode ignorado y confianza ciega en campos; verificar contratos.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-006 | Protección frente a SSRF y destinos dinámicos | Prioridad: Alta

Alcance: URLs o hosts influenciados por usuarios o datos externos.

Requisito propuesto: Resolver destinos mediante listas permitidas de esquemas, hosts, puertos y rutas; bloquear redes internas, metadatos de nube y redirecciones no autorizadas.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Rastrear origen de la URL hasta HttpClient; validar controles de red y resolución DNS, no solo comparación de cadenas.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-007 | Timeouts y control de recursos | Prioridad: Alta

Alcance: HttpClient, SDK y llamadas remotas.

Requisito propuesto: Definir timeouts, límites de tamaño de respuesta, concurrencia y cancelación; evitar esperas infinitas y agotamiento de conexiones o hilos.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar Timeout, CancellationToken, IHttpClientFactory y límites de lectura; confirmar valores contra requisitos de negocio.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-008 | Reintentos, backoff e idempotencia | Prioridad: Alta

Alcance: Fallos transitorios y operaciones mutables.

Requisito propuesto: Reintentar solo errores transitorios con retroceso exponencial y jitter; usar claves de idempotencia en operaciones no idempotentes y evitar tormentas de reintentos.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar políticas (por ejemplo Polly o resiliencia de Microsoft.Extensions) y método HTTP; no afirmar seguridad de reintento sin conocer el receptor.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-009 | Circuit breaker y degradación controlada | Prioridad: Media

Alcance: Dependencias críticas o inestables.

Requisito propuesto: Aplicar cortacircuitos, bulkheads, límites y comportamiento degradado definido para evitar propagación de fallos y bloqueo del sistema.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar políticas y comportamiento ante indisponibilidad; confirmar con pruebas de falla cuando sea posible.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-010 | Límites de tasa y cuotas | Prioridad: Alta

Alcance: APIs con límites o alto volumen.

Requisito propuesto: Respetar rate limits y Retry-After; aplicar control de tasa del lado cliente cuando corresponda y monitorear cuotas.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar manejo de 429, cabeceras de límite y bucles de llamadas; los límites reales dependen del proveedor.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-011 | Minimización y clasificación de datos | Prioridad: Alta

Alcance: Cargas útiles, parámetros y respuestas.

Requisito propuesto: Enviar solo datos necesarios y autorizados; no transmitir datos sensibles a terceros sin base, clasificación, contrato y aprobación; evitar datos personales en URL.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar DTO enviados, query strings y proveedor; la clasificación y aprobación requieren fuentes externas al código.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-012 | Registro seguro de integraciones | Prioridad: Alta

Alcance: Logs, trazas y telemetría de llamadas API.

Requisito propuesto: Registrar identificador de correlación, destino lógico, método, estado y latencia; enmascarar o excluir tokens, cabeceras sensibles, cuerpos y datos personales.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Buscar registro de HttpRequestMessage, cabeceras Authorization y cuerpos completos; revisar filtros de redacción.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-013 | Manejo seguro de errores remotos | Prioridad: Alta

Alcance: Respuestas 4xx, 5xx y excepciones de red.

Requisito propuesto: Mapear errores externos a errores internos estables, no propagar mensajes del tercero al usuario y distinguir fallos transitorios de funcionales.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar catch, mapeo de excepciones y respuesta final; confirmar que no haya éxito silencioso.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-014 | Contratos, versionamiento y compatibilidad | Prioridad: Alta

Alcance: Contratos OpenAPI, SDK y clientes generados.

Requisito propuesto: Versionar contratos, probar compatibilidad, controlar actualizaciones de SDK y detectar cambios incompatibles antes de despliegue.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar especificaciones, generación de clientes y pruebas de contrato; no suponer compatibilidad por usar la misma ruta.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-015 | Webhooks y callbacks entrantes | Prioridad: Alta

Alcance: Endpoints que reciben eventos de terceros.

Requisito propuesto: Autenticar el origen mediante firma, secreto compartido o mTLS; validar timestamp/nonce, proteger contra repetición, limitar tamaño y procesar de forma idempotente.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar comparación de firmas en tiempo constante, ventana temporal y deduplicación; la configuración del emisor requiere confirmación externa.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

API-016 | Pruebas de integración y simulación | Prioridad: Media

Alcance: Consumo de API en pruebas.

Requisito propuesto: Probar éxitos, errores, timeouts, respuestas malformadas, 401/403/429/5xx y reintentos con simuladores o servidores controlados; no usar datos productivos.

Cómo revisar: Seguir el flujo desde la configuración y la construcción de la solicitud hasta el procesamiento de la respuesta.

Evidencia y límites: Revisar pruebas y dobles de prueba; una prueba solo del camino exitoso no cubre la resiliencia.

Remediación sugerida: Aplicar el requisito con una política común reutilizable y pruebas automatizadas.

3. Ejemplo ilustrativo

Ejemplo didáctico parcial, con nombres ficticios. No demuestra por sí solo el cumplimiento de todas las reglas ni constituye código listo para producción.

```
// Ilustrativo: ademas de autenticar, validar acceso al recurso.
var cuenta = await repositorio.BuscarAsync(cuentaId, cancellationToken);
if (cuenta is null) return Results.NotFound();
var decision = await authorizationService.AuthorizeAsync(
    user, cuenta, "ConsultarCuenta");
if (!decision.Succeeded) return Results.Forbid();
return Results.Ok(new { cuenta.Estado });
```

Cliente de API con validación y manejo de errores

```
// Ilustrativo: cliente tipado con timeout, validacion y sin datos sensibles en logs.
public async Task<Result<EstadoCuentaDto>> ObtenerAsync(string cuentaId, CancellationToken ct)
{
    using var request = new HttpRequestMessage(HttpMethod.Get, $"v1/cuentas/{Uri.EscapeDataString(cuentaId)}");
    request.Headers.Authorization = await tokenProvider.GetBearerAsync(ct);
    request.Headers.Add("X-Correlation-Id", correlationContext.Id);
    using var response = await http.SendAsync(request, HttpCompletionOption.ResponseHeadersRead, ct);
    if (response.StatusCode == HttpStatusCode.TooManyRequests) return Result.Retry(response.Headers.RetryAfter);
    if (!response.IsSuccessStatusCode) return Result.Fail("Servicio externo no disponible.");
    if (response.Content.Headers.ContentType?.MediaType != "application/json") return Result.Fail("Respuesta invalida.")
    var dto = await response.Content.ReadFromJsonAsync<EstadoCuentaDto>(ct);
    return dto is { CuentaId.Length: > 0 } ? Result.Ok(dto) : Result.Fail("Contrato invalido.");
}
```

Registro de HttpClient con resiliencia

```
// Ilustrativo: HttpClient tipado y politicas de resiliencia (ajustar valores a la entidad).
builder.Services.AddHttpClient<IEstadoCuentaClient, EstadoCuentaClient>(c =>
{
    c.BaseAddress = new Uri(config["Integraciones:Cuentas:BaseUrl"]!);
    c.Timeout = TimeSpan.FromSeconds(10);
}).AddStandardResilienceHandler();
```

Recepción de webhook con firma y repetición controlada

```
// Ilustrativo: validar firma y ventana temporal antes de procesar un webhook.
var expected = ComputeHmac(secret, timestamp + "." + body);
if (!CryptographicOperations.FixedTimeEquals(expected, receivedSignature)) return Results.Unauthorized();
if (Math.Abs((DateTimeOffset.UtcNow - timestampValue).TotalMinutes) > 5) return Results.Unauthorized();
if (!await dedup.TryRegisterAsync(eventId, ct)) return Results.Ok(); // idempotente
```

4. Formato propuesto de resultado

Estado: incumple / cumple en alcance revisado / no verificable. Regla: ID y título. Evidencia: archivo, líneas y fragmento mínimo. Explicación: relación entre evidencia y requisito. Impacto: riesgo contextualizado. Recomendación: cambio concreto. Fuente: manual, versión y página. Si faltan archivos, permisos o configuración, indicarlo expresamente.

5. Referencias y alcance

Fuentes públicas de consulta; las reglas de este documento son una propuesta propia, no una reproducción ni certificación de conformidad con esas fuentes. Consultar versiones vigentes y compatibilidad aplicable.

OWASP: Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/

OWASP: guías de prevención: https://cheatsheetseries.owasp.org/

NIST: Secure Software Development Framework SP 800-218: https://csrc.nist.gov/pubs/sp/800/218/final

Microsoft: autorización basada en recursos: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/resourcebased

OWASP: API Security Top 10: https://owasp.org/API-Security/

Microsoft: IHttpClientFactory: https://learn.microsoft.com/en-us/dotnet/core/extensions/httpclient-factory

Microsoft: resiliencia HTTP en .NET: https://learn.microsoft.com/en-us/dotnet/core/resilience/http-resilience